



WPYEG • Edmonton WordPress Meetup

Better by Default

Secure defaults for every WordPress site.

a hands-on workshop · build the “sane-defaults” plugin

WordPress is open by default; hosts vary in what they close.

None of these are bugs — or quite the security risks popular opinion alleges. They are defaults chosen for maximum compatibility on a 20+ year-old web application.



Username leak

REST and author archives expose public author slugs that often resemble login names.



XML-RPC exposed

A venerable interop protocol Jetpack still uses. If you don't need it, it just adds attack surface.



Dead weight loads

Emoji scripts, version tags, and RSD links on every page. Do you need them?



Spam surface invites

Comments, pingbacks, and trackbacks are open by default. Legacy craft, or the heart of the open web?

The one idea to take home

A “default” is just an opinionated filter behind a toggle.

```
if ( wpyeg_defaults_enabled( 'restrict_rest_user_discovery' ) ) {  
    add_filter( 'rest_endpoints', $hide_users_endpoint );  
} // that's the whole pattern, repeated across the plugin
```

Hooks

WordPress is built to be interrupted at labelled moments (hooks) so you never edit core code.

Actions

A

“When you reach this moment, also DO this.”

```
add_action( 'init', fn );
```

Filters

F

“Before you use this value, let me CHANGE it first.”

```
add_filter( 'xmlrpc_enabled',  
    '__return_false' );
```



What wins when settings overlap?

1

wp-config.php constants

Load first. When core treats one as authoritative, plugin settings cannot override it.

2

Must-use plugins

Load before normal plugins, so their callbacks register first.

3

Normal plugins

Load in active_plugins order — PMP before BBD on this demo site.

4

Hook priority

Lower runs earlier; higher runs later. Ties keep registration order.

`effective behaviour = hard constants + every callback, in execution order`

Seven categories of defaults

1

Security

Make the attack surface smaller

2

Updates

A deliberate core & translation policy

3

Content

Close spam channels & info leaks

4

Admin UX

A calmer, faster, prettier dashboard

5

Login

Sessions & credentials

6

Branding

Own your login screen

7

Performance

Trim the fat

1

SECTION ONE

Security & Attack Surface

Every item here removes something an attacker can poke — usually in one line.

Restrict REST API user discovery

The `/wp/v2/users` endpoint hands out every author's login name to anyone — half of a brute-force guess, for free. Closing it for logged-out requests only keeps the editor and legit integrations working.

KEY	<code>restrict_rest_user_discovery</code>
DEFAULT	
yes	

```
add_filter( 'rest_endpoints', function ( $ep ) {  
    if ( ! is_user_logged_in() ) {  
        unset( $ep['/wp/v2/users'] );  
        unset(  
            $ep['/wp/v2/users/(?P<id>[\d]+)']  
        );  
    }  
    return $ep;  
} );
```


Lock REST to logged-in users (opt-in)

The sledgehammer version of the slide before: requiring auth for ALL REST calls stops anonymous scraping cold. It breaks anonymous REST — front-end blocks, embeds, search, outside integrations — so it ships off.

KEY	<code>disable_rest</code>
DEFAULT	<code>no</code>

```
add_filter( 'rest_authentication_errors',  
    function ( $result ) {  
        if ( ! empty( $result ) ) return $result;  
        if ( ! is_user_logged_in() ) {  
            return new WP_Error(  
                'rest_not_logged_in', 'Auth required.',  
                array( 'status' => 401 ) );  
        }  
        return $result;  
    } );
```

Lock XML-RPC down by category

XML-RPC is legitimate but aging — an extra surface, not a backdoor. We unplug unused method families while preserving the endpoint for integrations that need it.

KEY `xmlrpc allow keys + block_xmlrpc_endpoint`
DEFAULT
`no (all four)`

```
// each category off → remove its methods
add_filter( 'xmlrpc_methods', function ( $m ) {
    if ( ! allow( 'pingbacks' ) )
        unset( $m['pingback.ping'] );
    if ( ! allow( 'remote_publishing' ) )
        // drop wp.* metaWeblog.* mt.* blogger.*
        return $m;
} );

// multicall can't be filtered off (IXR re-adds it)
// → swap in a server that refuses it
add_filter( 'wp_xmlrpc_server_class', $refuse );
```

Keep Application Passwords available

This is an existing default we don't lock down. An Application Password is like a spare key cut for one app: hashed, per-application, revocable on its own — the safer REST credential, and the only one core accepts for REST Basic Auth.

KEY `disable_application_passwords`
DEFAULT
`no (available)`

```
// available by default -  
// prohibit only if opted in  
if ( wpyeg_defaults_enabled(  
    'disable_application_passwords' ) ) {  
    add_filter(  
        'wp_is_application_passwords_available',  
        '__return_false'  
    );  
}
```

Screen breaches without sending the password

NIST SP 800-63B-4 § 3.1.1.2: use 15+ characters, a blocklist, and no composition rules. BBD hashes the candidate locally, sends HIBP only the first 5 SHA-1 characters, and matches the remaining 35 locally. The password and full hash never leave WordPress; unavailable or invalid HIBP data fails open.

```
KEY      require_strong_passwords
DEFAULT
yes
```

```
$hash    = strtoupper( sha1( $pw ) );
$prefix  = substr( $hash, 0, 5 );
$suffix  = substr( $hash, 5 );

// HIBP receives $prefix and returns
// matching suffixes plus breach counts.
// BBD compares $suffix locally.
if ( response_contains( $suffix ) ) {
    reject( 'Seen in a breach.' );
}
// invalid or 128 KiB => fail open
```

Remove fingerprints, add headers

Two headers with no real downside ship on; framing is its own setting, because it is the only one that can break a working site. Hiding the version is obscurity, not hardening, so it ships off.

KEY **security_headers / frame_options**
DEFAULT
yes / SAMEORIGIN

```
remove_action( 'wp_head', 'wp_generator' );

add_filter( 'wp_headers', function ( $h ) {
    // only fill in what nothing else set
    if ( ! isset( $h['X-Content-Type-Options'] ) )
        $h['X-Content-Type-Options'] = 'nosniff';
    if ( ! isset( $h['Referrer-Policy'] ) )
        $h['Referrer-Policy'] =
            'strict-origin-when-cross-origin';
    return $h;
} );
// framing is its own setting - see the notes
```



2

SECTION TWO

Content & Public Surfaces

Close the spam funnels and the thin pages Google (and bots) love to crawl.

Disable comments, trackbacks & pingbacks

For many sites, comments are a spam magnet with little upside. Here we close them everywhere, hide existing threads, and drop the admin menu.

KEY `disable_comments / disable_pingbacks /`
`disable_self_pingbacks`
DEFAULT
`yes each`

```
add_filter( 'comments_open', '__return_false', 20 );
add_filter( 'pings_open',    '__return_false', 20 );
add_filter( 'comments_array',
            '__return_empty_array', 20 );

// + remove_post_type_support() on init
// + remove_menu_page('edit-comments.php')
// + drop the admin-bar comments node
```

Redirect author & attachment pages

Author archives expose the authors' usernames in the URL, and attachment pages are near-empty media wrappers. Both dilute SEO and are targets for trouble. Same hook, two conditions.

KEY `disable_author_archives /`
`redirect_attachment_pages`
DEFAULT
`yes / yes`

```
add_action( 'template_redirect', function () {  
    if ( is_author() ) {  
        wp_safe_redirect( home_url('/'), 301 );  
        exit;  
    }  
    if ( is_attachment() ) {  
        // parent post, else the FILE - never home  
    }  
} );
```


Disable the emoji script

WordPress core injects an emoji-detection script and inline CSS on every page load, plus a DNS-prefetch hint. Modern browsers render emoji natively, so this is pure dead weight.

KEY	<code>disable_emojis</code>
DEFAULT	
yes	

```
add_action( 'init', function () {  
    remove_action( 'wp_head',  
        'print_emoji_detection_script', 7 );  
    remove_action( 'wp_print_styles',  
        'print_emoji_styles' );  
    // ...admin + feed + mail variants too  
    add_filter( 'emoji_svg_url',  
        '__return_false' );  
} );
```



3

SECTION THREE

Admin UX & Login Sessions

Small quality-of-life defaults: a calmer dashboard and sensible session policy.

Faster search, quieter admin bar

Search the admin post list on a big site and WordPress reads every word of every post — like finding a book by reading the whole library. Title-only search checks just the spines, and it's far faster.

```
KEY      title_only_admin_search /  
frontend_admin_bar_behavior  
DEFAULT  
no / ''
```

```
// title-only admin search — narrow the COLUMNS  
add_filter( 'post_search_columns',  
    function ( $cols, $s, $q ) {  
        if ( is_admin() && $q->is_main_query() )  
            return array( 'post_title' );  
        return $cols;    // front-end untouched  
    }, 10, 3 );  
// hide bar for non-admins  
add_filter( 'show_admin_bar', fn( $s ) =>  
    current_user_can('manage_options') ? $s : false );
```

Right-size the login session

Click “Remember Me” and you stay logged in for 14 days. Cap that extended session, optionally shorten the regular one, or hide the checkbox entirely. One filter covers all three.

```
KEY      disable_remember_me / remember_me_days /  
session_regular_hours  
DEFAULT  
no / 5 / 0
```

```
add_filter( 'auth_cookie_expiration',  
function ( $exp, $uid, $remember ) {  
    if ( $remember ) {  
        return 5 * DAY_IN_SECONDS;  
    }  
    return 12 * HOUR_IN_SECONDS;  
}, 10, 3 );  
  
// DAY_IN_SECONDS: core time constant
```



4

SECTION FOUR

Branding & Performance

Own the login screen, then shave the last bit of weight off every page.

Own the login screen

The default WordPress logo sends users to wordpress.org. Removing, unlinking, or replacing it keeps the login screen organizationally consistent and prevents an unexpected external destination. BBD leaves it unchanged unless you opt in.

```
KEY      login_logo_behavior
DEFAULT
keep_default (keep / remove / unlink / replace)
```

```
// remove, unlink, or replace – a choice
add_action( 'login_head', $logo_css );

// any change points the link home
// (no separate toggle)
add_filter( 'login_headerurl', 'home_url' );
add_filter( 'login_headertext', fn() =>
    get_bloginfo( 'name' ) );
```

Throttle Heartbeat — and a default we deleted

Throttle Heartbeat to ease up on weak shared hosting. The more interesting half is the toggle that used to be here: WordPress 6.3 gave scripts a per-script loading strategy, so our blanket defer filter had to go.

```
KEY      throttle_heartbeat
DEFAULT
no (opt-in)
```

```
add_filter( 'heartbeat_settings', fn( $s ) => {
    $s['interval'] = 60; return $s;
} );

// Deferring is NOT a setting here. Since WP 6.3:
wp_enqueue_script( 'front', $src, array(), '1.0',
    array( 'strategy' => 'defer' ) );
```

Three things a plugin can't toggle

Some defaults live in wp-config.php, above the plugin layer, because they must load before plugins do. Document them as manual steps in your onboarding checklist.

```
define( 'DISALLOW_FILE_EDIT', true ); // no in-dashboard code editor
define( 'AUTOSAVE_INTERVAL', 120 ); // gentler autosave (seconds)
define( 'WP_POST_REVISIONS', 10 ); // cap revision-table bloat
```

1

Kills the theme/plugin editor

A stolen admin login can't rewrite your PHP.

2

Writes to the DB less often

Fewer autosave revisions during long edits.

3

Keeps revisions in check

Ten per post instead of unbounded growth.

How the plugin is built

One array is the whole map. Read it and you understand the plugin.

1 `schema()`

One array: every setting, its default, type & group. The single source of truth.



2 `settings page`

Loops the schema to render toggles under Settings → Better by Default.



3 `bootstrap()`

For each ENABLED key, wires its `add_filter` / `add_action` to the right hook.

```
$stored = get_option('wpyeg_better_by_default'); // read once
foreach ( wpyeg_defaults_schema() as $key => $field ) { /* render + wire */ }
```



Hands-on: install & flip switches

1

Upload the plugin

Plugins → Add New → Upload Plugin → choose sane-defaults.zip → Activate.

2

Open the settings

Settings → Better by Default; every toggle grouped by category.

3

Verify a default

Visit `/wp-json/wp/v2/users` logged out → 401 or empty, not a list of usernames.

4

Toggle & re-check

Flip a switch off, reload, watch the behaviour change.

```
prefer the terminal? wp plugin install ./sane-defaults.zip --activate
```

Your turn

Add one new default to the plugin

Goal: disable the WordPress dashboard “Welcome” panel. Two small edits — no new settings-page code.

```
// 1) add a schema entry in wpyeg_defaults_schema()
'hide_welcome_panel' => array(
    'default' => 'yes', 'type' => 'toggle', 'group' => 'ux',
    'label' => 'Hide dashboard welcome panel',
),

// 2) wire it inside wpyeg_defaults_bootstrap()
if ( wpyeg_defaults_enabled( 'hide_welcome_panel' ) ) {
    remove_action( 'welcome_panel', 'wp_welcome_panel' );
}
```

Schema map — security surfaces and credentials

Exact keys and defaults from `wpyeg_defaults_schema()`.

Setting key or owner	Default	Core hook / authority
<code>restrict_rest_user_discovery</code>	<code>yes</code>	<code>rest_endpoints</code>
<code>disable_rest</code>	<code>no</code>	<code>rest_authentication_errors</code>
<code>xmlrpc_allow_pingbacks</code>	<code>no</code>	<code>xmlrpc_methods</code> / <code>headers</code>
<code>xmlrpc_allow_remote_publishing</code>	<code>no</code>	<code>xmlrpc_methods</code> / <code>discovery</code>
<code>xmlrpc_allow_multicall</code>	<code>no</code>	<code>wp_xmlrpc_server_class</code>
<code>block_xmlrpc_endpoint</code>	<code>no</code>	<code>template_redirect</code>
<code>disable_application_passwords</code>	<code>no</code>	<code>wp_is_application_passwords_available</code>
<code>require_strong_passwords</code>	<code>yes</code>	server-side password validation

Schema map — security policy and updates

Plugin defaults and the policies intentionally left to other layers.

Setting key or owner	Default	Core hook / authority
remove_version	no	wp_head / the_generator
security_headers	yes	wp_headers
frame_options	SAMEORIGIN	wp_headers
disable_ai_connectors	yes	wp_supports_ai / Connectors screen
core_update_policy	minor	automatic core-update filters
Translation files	inherit	WordPress / host / fleet tooling
Plugin and theme code	per-item	WordPress per-item choices
WP_AUTO_UPDATE_CORE	operator	wp-config.php wins

Schema map — content and everyday UX

The schema group is authoritative; emoji removal is a Content setting.

Setting key or owner	Default	Core hook / authority
disable_comments	yes	comments, UI, post-type support
disable_pingbacks	yes	default ping options
disable_self_pingbacks	yes	pre_ping
disable_author_archives	yes	template_redirect
redirect_attachment_pages	yes	template_redirect
disable_emojis	yes	init removes emoji assets
title_only_admin_search	no	post_search_columns
frontend_admin_bar_behavior	' '	show_admin_bar

Schema map — login, branding, and performance

Schema keys live in one option array; constants remain above plugins.

Setting key or owner	Default	Core hook / authority
disable_remember_me	no	login UI / cookie expiration
remember_me_days	5	auth_cookie_expiration
session_regular_hours	0	auth_cookie_expiration
login_logo_behavior	keep_default	login header presentation
throttle_heartbeat	no	Heartbeat settings / enqueue
wpyeg_better_by_default	array	the only wp_options row
DISALLOW_FILE_EDIT	manual	wp-config.php
revisions / autosave	manual	wp-config.php constants



Thanks, WPYEG!

Set your defaults wisely.

Files: `sane-defaults.zip` · `wordpress-default-settings.md`

Questions? License GPL-3.0-or-later